

实验二、 Open vSwitch 应用实践

一、 教学目的与学时建议

1. 能够对 Open vSwitch 进行基本操作
2. 能够运用 Open Flow 命令对交换机进行管理
3. 能够使用 Floodlight 控制 OVS 交换机
4. 建议：2 学时

二、 技术与知识背景

1. Open vSwitch 概述

Open vSwitch (下面简称为 OVS) 是由 Nicira Networks 主导的, 运行在虚拟化平台 (例如 KVM, Xen) 上的虚拟交换机。在虚拟化平台上, OVS 可以为动态变化的端点提供 2 层交换功能, 很好的控制虚拟网络中的访问策略、网络隔离、流量监控等等。

OVS 遵循 Apache 2.0 许可证, 能同时支持多种标准的管理接口和协议。OVS 也提供了对 OpenFlow 协议的支持, 用户可以使用任何支持 OpenFlow 协议的控制器对 OVS 进行远程管理控制。

在 OVS 中, 有几个非常重要的概念:

① Bridge: Bridge 代表一个以太网交换机 (Switch), 一个主机中可以创建一个或者多个 Bridge 设备。

② Port: 端口与物理交换机的端口概念类似, 每个 Port 都隶属于一个 Bridge。

③ Interface: 连接到 Port 的网络接口设备。在通常情况下, Port 和 Interface 是一一对一的关系, 只有在配置 Port 为 bond 模式后, Port 和 Interface 是一对多的关系。

④ Controller: OpenFlow 控制器。OVS 可以同时接受一个或者多个 OpenFlow 控制器的管理。

⑤ Datapath: 在 OVS 中, datapath 负责执行数据交换, 也就是把从接收端口收到的数据包在流表中进行匹配, 并执行匹配到的动作。

⑥ Flow table: 每个 datapath 都和一个“flow table”关联, 当 datapath 接收到数据之后, OVS 会在 flow table 中查找可以匹配的 flow, 执行对应的操作, 例如转发数据到另外的端口。

2. OVS 各模块的简要介绍

- **ovs-vswitchd**: 主要模块, 实现 switch 的 daemon, 包括一个支持流交换的 Linux 内核模块;

- **ovsdb-server**: 轻量级数据库服务器, 提供 ovs-vswitchd 获取配置信息;

- **ovs-dpctl**: 用来配置 switch 内核模块;

一些 Scripts and specs 辅助 OVS 安装在 Citrix XenServer 上, 作为默认 switch;

- **ovs-vsctl**: 查询和更新 ovs-vswitchd 的配置;

- **ovs-appctl**: 发送命令消息, 运行相关 daemon。

OVS 提供了支持 OpenFlow 的特性实现, 包括:

- **ovs-ofctl**: 查询和控制 OpenFlow 交换机和控制器;

- **ovs-pki**: OpenFlow 交换机创建和管理公钥框架;

- **ovs-tcpundump**: tcpdump 的补丁, 解析 OpenFlow 的消息。

3. 基于 Open vSwitch 的 Open Flow 实践

OpenFlow 是用于管理交换机流表的协议, 在 OpenFlow 的白皮书中, Flow 被定义为某个特定的网络流量。例如, 一个 TCP 连接就是一个 Flow, 或者从某个 IP 地址发出来的数据包, 都可以被认为是一个 Flow。支持 OpenFlow 协议的交换机应该包括一个或者多个流表, 流表中的条目包含: 数据包头的信息、匹配成功后要执行的指令和统计信息。

当数据包进入 OVS 后, 会将数据包和流表中的流表项进行匹配, 如果发现了匹配的流表项, 则执行该流表项中的指令集。相反, 如果数据包在流表中没有发现任何匹配, OVS 会通过控制通道把数据包发到 OpenFlow 控制器中。

在 OVS 中, ovs-ofctl 则是 OVS 提供的命令行工具。在没有配置 OpenFlow 控制器的模式下, 用户可以使用 ovs-ofctl 命令通过 OpenFlow 协议去连接 OVS, 创建、修改或删除 OVS 中的流表项, 并对 OVS 的运行状况进行动态监控。流表项作为 ovs-ofctl 的参数, 采用如下的格式: 字段=值。如果有多个字段, 可以用逗号或者空格分开。一些常用的字段列举如表 1 所示。

表 1. 流表项匹配字段

字段名称	说明
in_port=port	传递数据包的端口的 OpenFlow 端口编号

dl_vlan=vlan	数据包的 VLAN Tag 值，范围是 0-4095，0xffff 代表不包含 VLAN Tag 的数据包
dl_src=<MAC> dl_dst=<MAC>	匹配源或者目标的 MAC 地址 01:00:00:00:00:00/01:00:00:00:00:00 代表广播地址 00:00:00:00:00:00/01:00:00:00:00:00 代表单播地址
dl_type=ethertype	匹配以太网协议类型，其中： dl_type=0x0800 代表 IPv4 协议 dl_type=0x086dd 代表 IPv6 协议 dl_type=0x0806 代表 ARP 协议
nw_src=ip[/netmask] nw_dst=ip[/netmask]	当 dl_type=0x0800 时，匹配源或者目标的 IPv4 地址，可以使 IP 地址或者域名
nw_proto=proto	和 dl_type 字段协同使用。 当 dl_type=0x0800 时，匹配 IP 协议编号 当 dl_type=0x086dd 代表 IPv6 协议编号
table=number	指定要使用的流表的编号，范围是 0-254。在不指定的情况下，默认值为 0。通过使用流表编号，可以创建或者修改多个 Table
reg<idx>=value[/mask]	交换机中的寄存器的值。当一个数据包进入交换机时，所有的寄存器都被清零，用户可以通过 Action 的指令修改寄存器中的值

对于 add-flow, add-flows 和 mod-flows 这三个命令，还需要指定要执行的动作：
actions=[target][,target...]

一个流规则中可能有多个动作，按照指定的先后顺序执行。

常见的操作有：

- output:port: 输出数据包到指定的端口。port 是指端口的 OpenFlow 端口编号
- mod_vlan_vid: 修改数据包中的 VLAN tag
- strip_vlan: 移除数据包中的 VLAN tag
- mod_dl_src/ mod_dl_dest: 修改源或者目标的 MAC 地址信息
- mod_nw_src/mod_nw_dst: 修改源或者目标的 IPv4 地址信息
- resubmit:port: 替换流表的 in_port 字段，并重新进行匹配
- load:value->dst[start..end]: 写数据到指定的字段

4. OVS 常用操作

- 1) 添加网桥：ovs-vsctl add-br 交换机名
- 2) 删除网桥：ovs-vsctl del-br 交换机名
- 3) 添加端口：ovs-vsctl add-port 交换机名 端口名（网卡名）
- 4) 删除端口：ovs-vsctl del-port 交换机名 端口名（网卡名）

- 5) 连接控制器: `ovs-vsctl set-controller 交换机名 tcp:IP 地址:端口号`
- 6) 断开控制器: `ovs-vsctl del-controller 交换机名`
- 7) 列出所有网桥: `ovs-vsctl list-br`
- 8) 列出网桥中的所有端口: `ovs-vsctl list-ports 交换机名`
- 9) 列出所有挂接到网卡的网桥: `ovs-vsctl port-to-br 端口名 (网卡名)`
- 10) 查看 open vswitch 的网络状态: `ovs-vsctl show`
- 11) 查看 Open vSwitch 中的端口信息 (交换机对应的 dpid, 以及每个端口的 OpenFlow 端口编号, 端口名称, 当前状态等等): `ovs-ofctl show 交换机名`
- 12) 修改 dpid: `ovs-vsctl set bridge 交换机名 other_config:datapath-id=新 DPID`
- 13) 修改端口号: `ovs-vsctl set Interface 端口名 ofport_request=新端口号`
- 14) 查看交换机中的所有 Table: `ovs-ofctl dump-tables ovs-switch`
- 15) 查看交换机中的所有流表项: `ovs-ofctl dump-flows ovs-switch`
- 16) 删除编号为 100 的端口上的所有流表项:
`ovs-ofctl del-flows ovs-switch "in_port=100"`
- 17) 添加流表项 (以 “添加新的 OpenFlow 条目, 匹配从端口 1 收到的数据包, 动作是向端口 2 转发为例”):
`sh ovs-ofctl add-flow s1 in_port=1,actions= output:2 "`
- 18) 查看 OVS 的版本信息: `ovs-appctl - version`
- 19) 查看 OVS 支持的 OpenFlow 协议的版本: `ovs-ofctl - version`

三、 基础技能

1. 内容与任务

在该实验中, 我们会创建一个不连接到任何控制器的 OVS 交换机, 创建 OVS 交换机的端口 p0, 设置端口 p0 的 OpenFlow 端口编号, 设置端口类型为 “internal”。为了避免网络接口上的地址和本机已有网络地址冲突, 我们需要创建一个虚拟网络空间 ns0, 把 p0 端口移入网络空间 ns0, 并配置 IP 地址。按照上述步骤再创建虚拟网络空间 ns1 和端口 p1, 测试其连通性。相应的任务分解见表 2。

表 2. “Open vSwitch 应用实践”的任务分解

任务分解	任务描述
OVS 交换机的创建	通过 <code>ovs-octl</code> 命令, 创建一个 OVS 交换机
端口 p0 的配置	创建端口 p0,p1, 设置端口 p0,p1 的 OpenFlow 配置端口编号, 并设置接口类型

虚拟网络空间的配置	创建一个虚拟网络空间 ns0，把 p0 端口移入网络空间 ns0，并配置 IP 地址；创建一个虚拟网络空间 ns1，把 p1 端口移入网络空间 ns1，并配置 IP 地址。
查看 OVS 交换机	查看 OVS 交换机的端口信息
使用 ovs-ofctl 创建并测试 OpenFlow 命令	查看 Open vSwitch 端口信息，查看交换机中的所有 Table，查看交换机中的所有流表项，端口互 ping 测试。

2. 规划与准备

要求学生根据上述任务要求完成必要的前期规划，表 3 给出一种参考，学生可以自己提出规划。

表 3 “Open vSwitch 应用实践”的相关规划

任务分解	规划要点	参考建议
OVS 交换机的创建	OVS 交换机的名字设置	使用“ovs-octl”命令，创建一个名为 ovs-switch 的 OVS 交换机
端口 p0 的配置	确定网络接口设备的类型，设置端口编号	创建端口 p0,p1，设置端口 p0, p1 的 OpenFlow 端口编号分别为 100、101，并设置接口类型为“internal”
虚拟网络空间的配置	将以创建的端口移入网络空间，设置配置 IP	创建一个虚拟网络空间 ns0，把 p0 端口移入网络空间 ns0，并配置 IP 地址为 192.168.1.100；创建一个虚拟网络空间 ns1，把 p1 端口移入网络空间 ns1，而 p1 的 IP 地址为 192.168.1.101
查看 OVS 交换机	查看所建的 OVS 交换机的配置	查看 OVS 交换机的端口信息，以便于之后的测试 OpenFlow 命令
使用 ovs-ofctl 创建并测试 OpenFlow 命令	熟悉 OpenFlow 命令，	从端口 p0 (192.168.1.100) 发送测试数据到端口 p1 (192.168.1.101)，通过 ovs-ofctl 命令对交换机的 Table、流表项、端口信息进行查看

四、基础技能的配置与操作要点

1. 交换机创建和端口 p0 的配置

创建一个新的 OVS 交换机：

\$ ovs-vsctl add-br [名称]

如：sudo ovs-vsctl add-br ovs-switch

创建一端口 p0，设置端口 p0 的 OpenFlow 端口编号为 100，如果在创建端口的时候没有指定 OpenFlow 端口编号，OVS 会自动生成一个。

\$ sudo ovs-vsctl add-port ovs-switch p0

set Interface p0 of port_request=100

设置网络接口设备的类型为“internal”。对于 internal 类型的网络接口，OVS 会同

时在 Linux 系统中创建一个可以用来收发数据的模拟网络设备。我们可以为这个网络设备配置 IP 地址、进行数据监听等等。查看结果。

```
$ sudo ovs-vsctl set Interface p0 type=internal
$ sudo ethtool -i p0
```

2. 虚拟网络空间的配置

为了避免网络接口上的地址和本机已有网络地址冲突,我们可以创建一个虚拟网络空间 ns0, 把 p0 接口移入网络空间 ns0, 并配置 IP 地址为 192.168.1.100。

```
$ sudo ip netns add ns0
$ sudo ip link set p0 netns ns0
$ sudo ip netns exec ns0 ip addr add 192.168.1.100/24 dev p0
$ sudo ip netns exec ns0 ifconfig p0 promisc up
```

3. 查看 OVS 交换机

通过 `sudo ovs-vsctl show` 命令查看 OVS 交换机的信息。

```
ubuntu@sdnhubvm:~[18:19]$ sudo ovs-vsctl show
873c293e-912d-4067-82ad-d1116d2ad39f
```

```
Bridge ovs-switch
  Port "p1"
    Interface "p1"
      type: internal
  Port ovs-switch
    Interface ovs-switch
      type: internal
  Port "p0"
    Interface "p0"
      type: internal
ovs_version: "2.3.90"
```

4. 使用 ovs-ofctl 创建并测试 OpenFlow 命令

查看 Open vSwitch 中的端口信息。从输出结果中,可以获得交换机对应的 datapath ID (dpid), 以及每个端口的 OpenFlow 端口编号, 端口名称, 当前状态。

- ①查看交换机中的所有 Table: `$ sudo ovs-ofctl dump-tables ovs-switch`
- ②查看交换机中的所有流表项: `$ sudo ovs-ofctl dump-flows ovs-switch`
- ③查看交换机上的端口信息: `$ sudo ovs-ofctl show ovs-switch`
- ④两个端口相互测试: `$ sudo ip netns exec ns0 ping 192.168.1.101`

五、进阶技能

1. 任务与要求

通过创建拓扑连接 floodlight 控制, 通过控制器的图像化界面, 编辑交换机流表规则来

控制数据转发。自主完成以下任务。

- 1) 在虚拟机环境下，打开终端运行 Floodlight；提示：

```
cd floodlight // 进入 floodlight 文件夹
```

```
java -jar target/floodlight.jar //运行 floodlight 控制器
```

- 2) 创建 1 个交换机连接 4 台主机的拓扑，并连接控制器。提示：

```
sudo mn --topo single,4 --controller remote,ip=127.0.0.1,port=6653
```

- 3) 打开 floodlight 图像化界面并查看网页中的 topo。提示：

```
localhost:8080/ui/pages/index.html //在虚拟机的浏览器中访问控制器的 web UI
```

- 4) 在 mininet 中执行 pingall 命令，在网页中查看交换机 s1 的流表。
- 5) 在网页中给交换机中添加一条流表项，使得 h1 不能访问其他所有主机。并在 mininet 中进行 pingall 测试。注意：表的序号是从 0 开始的。
- 6) 删除之前添加的流表项，在 mininet 中进行 pingall 测试。